Sri Lanka Institute of Information Technology

IT4060 - Machine Learning

[2026/JAN]

Assignment 01

Heart Disease Prediction using Supervised Machine Learning Algorithms

Name with Initials	Registration Number	Email
JEEWANTHI K.H	IT22171924	it22171924@my.sliit.lk
RANDENIYA T.D.P	IT22276278	it22276278@my.sliit.lk
MALMITH H.G.P.Y	IT22554604	it22554604@my.sliit.lk

Table of Contents

1. Introduction 3

2. Methodology 3

3. Dataset Description 3

4. Algorithm Justification 4

5. Critical Analysis 4

    5.1 Model Comparison 4

    5.2 Improvements 5

    5.3 Future Work 5

6. Discussion 5

7. Individual Contribution 6

8. Conclusion 6

9. Appendix 6

    9.1 Logistic Regression Code 7

    9.2 Decision Tree Code 12

    9.3 Random Forest Code 14

    9.4 Support Vector Machine Code 19

    9.5 Main Notebook Code 21

1. Introduction

Machine learning is widely used in healthcare to identify patterns in patient data and support early disease prediction. In this project, the selected real-world problem is heart disease prediction using clinical attributes collected from patients.

The goal of the study is to classify whether a patient is likely to have heart disease or not by applying four supervised learning algorithms: Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM).

The uploaded notebooks show that the project uses the same heart disease classification problem across multiple models, evaluates the models using standard classification metrics, and includes optimization and comparison steps for improved analysis.

2. Methodology

The overall workflow followed in the submitted notebooks consists of dataset loading, exploratory inspection, preprocessing, train-test splitting, feature scaling where required, model training, prediction, evaluation, and optimization.

The heart disease dataset was first loaded from a CSV file and inspected for shape, columns, data types, class distribution, and missing values. One of the notebooks also removed duplicate rows as a preprocessing step. The target variable is binary, where 1 represents the presence of heart disease and 0 represents the absence of heart disease.

After preprocessing, the data was divided into training and testing subsets. Models sensitive to feature magnitude, such as Logistic Regression and SVM, used StandardScaler to normalize the feature space. Each algorithm was then trained on the training set and evaluated on the testing set using measures such as accuracy, precision, recall, F1 score, confusion matrix, and ROC-AUC.

Hyperparameter tuning was applied in multiple notebooks through GridSearchCV. This made it possible to compare baseline models with optimized versions and to identify parameter settings that better match the dataset.

3. Dataset Description

The project uses a publicly available heart disease dataset in CSV format. Based on the notebook outputs, the raw dataset contains 1,025 records and 14 columns, of which 13 are predictive features and 1 is the target label.

Dataset link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

The class distribution shown in the uploaded notebooks is balanced enough for binary classification, with 526 positive cases and 499 negative cases. The dataset contains no missing values according to the preprocessing checks performed in the notebooks.

Important attributes used in the dataset are listed below:

Feature	Description
age	Age of the patient in years
sex	Biological sex of the patient
cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar
restecg	Resting electrocardiographic result
thalach	Maximum heart rate achieved
exang	Exercise-induced angina
oldpeak	ST depression induced by exercise relative to rest
slope	Slope of the peak exercise ST segment
ca	Number of major vessels colored by fluoroscopy
thal	Thalassemia-related test result
target	Binary class label: 1 = heart disease, 0 = no heart disease

4. Algorithm Justification

Logistic Regression - Logistic Regression is an appropriate baseline model for binary classification. It is simple, interpretable, computationally efficient, and useful for understanding how strongly each feature is associated with the target class.

Decision Tree - Decision Tree is suitable because it can model non-linear relationships and is easy to interpret visually. It provides clear decision rules and can highlight the most important predictors in the dataset.

Random Forest - Random Forest is an ensemble method built from multiple decision trees. It usually improves generalization, reduces overfitting, and provides robust performance when compared with a single tree.

Support Vector Machine - SVM is effective for high-dimensional classification problems and can learn complex decision boundaries. With proper scaling and hyperparameter tuning, it can achieve strong predictive performance.

5. Critical Analysis

5.1 Model Comparison

Table 1 summarizes the reported outputs available in the uploaded notebooks. The SVM appendix is included as source code text, but a final SVM notebook with measured results was not attached in the current upload set; therefore its metric cells should be updated before final submission.

Model	Reported Accuracy	Key Observation	Availability
Logistic Regression	77.05%	Good recall and interpretability; optimized version slightly reduced test accuracy.	Notebook uploaded
Decision Tree	98.54%	Very strong predictive performance; optimized max_depth = 10.	Notebook uploaded
Random Forest	100.00%	Best reported test accuracy; cross-validation accuracy reported as 98.54%.	Notebook uploaded
Support Vector Machine	To be inserted	Code pipeline prepared; final measured metrics should be added from the submitted SVM notebook.	Code appendix included

Among the available reported outputs, Random Forest produced the highest test accuracy, while Decision Tree also performed strongly. Logistic Regression gave a lower accuracy but remained valuable as an interpretable baseline model. The available notebook evidence therefore suggests that tree-based models fit this dataset more effectively than the linear baseline.

5.2 Improvements

Use one consistent preprocessing pipeline for all four models so that the final comparison is perfectly fair.

Apply hyperparameter tuning to every model using the same cross-validation strategy.

Use feature selection or feature importance analysis to remove weak predictors and simplify the final model.

Evaluate the models with additional metrics such as ROC-AUC and confusion-matrix-based error analysis for all algorithms.

5.3 Future Work

Integrate the best-performing classifier into a simple web or desktop prediction application.

Test the models on additional heart disease datasets to evaluate robustness and generalization.

Explore ensemble boosting methods such as XGBoost or Gradient Boosting for improved performance.

Standardize model comparison through a single main notebook that trains every model on the same split and reports results in one table.

6. Discussion

The project demonstrates that supervised machine learning can be used effectively for heart disease prediction using structured clinical features. The reported notebook results indicate that models capable of learning non-linear relationships, particularly Decision Tree and Random Forest, performed extremely well on this dataset.

Logistic Regression remained useful because of its interpretability and strong recall, even though its overall accuracy was lower than the best tree-based models. The optimization results also show an important lesson: hyperparameter tuning does not automatically guarantee a higher test accuracy, and the final tuned model must always be validated carefully.

The notebooks also highlight the importance of preprocessing. Data quality checks, duplicate handling, class balance inspection, and appropriate feature scaling all contribute to building reliable models and meaningful comparisons.

7. Individual Contribution

Update the following table with the actual names and registration numbers before submission.

Member	Contribution	Status
Member 1	Logistic Regression + preprocessing	Update details
Member 2	Decision Tree	Update details
Member 3	Random Forest	Update details
Member 4	Support Vector Machine + final comparison	Update details

8. Conclusion

This project applied multiple supervised machine learning algorithms to the problem of heart disease prediction using a real-world dataset. The submitted work demonstrates a full machine learning workflow that includes preprocessing, model development, evaluation, optimization, and comparative analysis.

Based on the reported notebook outputs currently available, Random Forest achieved the best predictive performance, while Decision Tree also produced excellent results. Logistic Regression served as a clear and interpretable baseline. The final report can be strengthened further by adding the measured SVM results into the comparison table and final discussion.

Overall, the project satisfies the core objective of showing how machine learning algorithms can be used to support practical decision-making in a healthcare-related classification problem.

9. Appendix

All source code below is included as plain text, not screenshots, in accordance with the assignment submission instructions.

9.1 Logistic Regression Code

```
import os
os.getcwd()

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score, roc_curve

data = pd.read_csv("heart.csv")
data.head()

data.shape

data.columns

data.info()

data.describe()

print("Preprocess part of the Dataset")

data.isnull().sum()

data.duplicated().sum()

data = data.drop_duplicates()

sns.countplot(x="target", data=data)
plt.title("Target Distribution")

plt.savefig("results/target_distribution.png", dpi=300)

plt.show()

data.shape

data["target"].value_counts()

data["target"].value_counts(normalize=True) * 100

sns.countplot(x="target", data=data)
plt.title("Target Class Distribution")
plt.show()

corr = data.corr()
corr

plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=False, cmap="coolwarm")
plt.title("Feature Correlation Heatmap")

plt.savefig("results/lr_correlation_heatmap.png", dpi=300)

plt.show()

data.hist(figsize=(12,10))
plt.show()

plt.figure(figsize=(12,6))
sns.boxplot(data=data)
plt.xticks(rotation=90)
plt.title("Boxplot of Features")
plt.show()

X = data.drop("target", axis=1)
```

```
y = data["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape, y_train.shape, y_test.shape

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

X_train_scaled_df.head()

print("===== Training Logistic Regression Model =====")

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train_scaled, y_train)

lr_pred = lr_model.predict(X_test_scaled)

print("===== Logistic Regression Evaluation =====")

print("Accuracy:", accuracy_score(y_test, lr_pred))
print("Precision:", precision_score(y_test, lr_pred))
print("Recall:", recall_score(y_test, lr_pred))
print("F1 Score:", f1_score(y_test, lr_pred))

# Generate confusion matrix
cm = confusion_matrix(y_test, lr_pred)

# Print confusion matrix
print("Confusion Matrix:")
print(cm)

# Plot confusion matrix as heatmap
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

# Save image
plt.savefig("results/logistic_confusion_matrix.png", dpi=300)

plt.show()

# Print classification report
print("Classification Report:")
print(classification_report(y_test, lr_pred))

# Get probability scores for positive class
lr_prob = lr_model.predict_proba(X_test_scaled)[:, 1]

# Calculate ROC-AUC score
auc_score = roc_auc_score(y_test, lr_prob)

print("ROC-AUC Score:", auc_score)

# Calculate false positive rate and true positive rate
fpr, tpr, thresholds = roc_curve(y_test, lr_prob)

# Plot ROC curve
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label="Logistic Regression")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")

# Save image
plt.savefig("results/logistic_roc_curve.png", dpi=300)
```

```
plt.legend()
plt.show()

# Import GridSearchCV
from sklearn.model_selection import GridSearchCV

# Define hyperparameter values to test
param_grid = {
    "C": [0.01, 0.1, 1, 10, 100],
    "solver": ["liblinear", "lbfgs"]
}

# Create GridSearchCV object
grid_search = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000),
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)

# Train grid search on scaled training data
grid_search.fit(X_train_scaled, y_train)

# Print best parameter values
print("Best Parameters:", grid_search.best_params_)
print("Best Cross Validation Score:", grid_search.best_score_)

# Get the best model found by GridSearchCV
best_lr_model = grid_search.best_estimator_

# Predict test data using best logistic regression model
best_lr_pred = best_lr_model.predict(X_test_scaled)

# Get probability scores from optimized model
best_lr_prob = best_lr_model.predict_proba(X_test_scaled)[:, 1]

# Print evaluation metrics for optimized model
print("===== Optimized Logistic Regression Evaluation =====")
print("Accuracy:", accuracy_score(y_test, best_lr_pred))
print("Precision:", precision_score(y_test, best_lr_pred))
print("Recall:", recall_score(y_test, best_lr_pred))
print("F1 Score:", f1_score(y_test, best_lr_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, best_lr_prob))

# Create confusion matrix for optimized model
best_cm = confusion_matrix(y_test, best_lr_pred)

# Plot optimized confusion matrix
plt.figure(figsize=(6,4))
sns.heatmap(best_cm, annot=True, fmt="d", cmap="Greens")
plt.title("Confusion Matrix - Optimized Logistic Regression")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()

# Print classification report for optimized model
print("Optimized Classification Report:")
print(classification_report(y_test, best_lr_pred))

# Calculate ROC curve values for optimized model
best_fpr, best_tpr, best_thresholds = roc_curve(y_test, best_lr_prob)

original_acc = accuracy_score(y_test, lr_pred)
optimized_acc = accuracy_score(y_test, best_lr_pred)

plt.bar(["Original", "Optimized"], [original_acc, optimized_acc])
plt.title("Model Improvement")

plt.savefig("results/logistic_optimization.png")
plt.show()

# Plot ROC curve for optimized model
plt.figure(figsize=(6,4))
plt.plot(best_fpr, best_tpr, label="Optimized Logistic Regression")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Optimized Logistic Regression")
plt.legend()
plt.show()

# Compare original and optimized model results
print("===== Model Comparison =====")
print("Original Accuracy:", accuracy_score(y_test, lr_pred))
print("Optimized Accuracy:", accuracy_score(y_test, best_lr_pred))
print("Original F1 Score:", f1_score(y_test, lr_pred))
print("Optimized F1 Score:", f1_score(y_test, best_lr_pred))
print("Original ROC-AUC:", roc_auc_score(y_test, lr_prob))
print("Optimized ROC-AUC:", roc_auc_score(y_test, best_lr_prob))

# 1. Save predictions (Actual vs Predicted)
results_df = pd.DataFrame({
    "Actual": y_test,
    "Predicted": lr_pred
})

results_df.to_csv("logistic_regression_predictions.csv", index=False)
print("Saved: logistic_regression_predictions.csv")

# 2. Save prediction probabilities (optional but good for report)
lr_prob = lr_model.predict_proba(X_test_scaled)[:, 1]

prob_df = pd.DataFrame({
    "Actual": y_test,
    "Predicted": lr_pred,
    "Probability": lr_prob
})

prob_df.to_csv("logistic_regression_probabilities.csv", index=False)
print("Saved: logistic_regression_probabilities.csv")

# 2. Save prediction probabilities (optional but good for report)
lr_prob = lr_model.predict_proba(X_test_scaled)[:, 1]

prob_df = pd.DataFrame({
    "Actual": y_test,
    "Predicted": lr_pred,
    "Probability": lr_prob
})

prob_df.to_csv("logistic_regression_probabilities.csv", index=False)
print("Saved: logistic_regression_probabilities.csv")

# 3. Save evaluation metrics
metrics_df = pd.DataFrame([{
    "Model": "Logistic Regression",
    "Accuracy": accuracy_score(y_test, lr_pred),
    "Precision": precision_score(y_test, lr_pred),
    "Recall": recall_score(y_test, lr_pred),
    "F1 Score": f1_score(y_test, lr_pred)
}])

metrics_df.to_csv("logistic_regression_metrics.csv", index=False)
print("Saved: logistic_regression_metrics.csv")

# 4. Save optimized model results (if you used GridSearchCV)
try:
    optimized_df = pd.DataFrame({
        "Actual": y_test,
        "Predicted_Optimized": best_lr_pred
    })

    optimized_df.to_csv("optimized_logistic_regression_predictions.csv", index=False)
    print("Saved: optimized_logistic_regression_predictions.csv")

except:
    print("Optimized model not found, skipping optimized CSV.")

# Final conclusion of this notebook
print("Conclusion:")
print("This notebook applied Logistic Regression to the heart disease dataset.")
print("The dataset was preprocessed by checking missing values, removing duplicates, splitting the
data, and scaling the features.")
```

```
print("The model was trained and evaluated using Accuracy, Precision, Recall, F1 Score, Confusion
Matrix, Classification Report, and ROC-AUC.")
print("Finally, GridSearchCV was used to tune the model and improve performance.")
print("This completed the Logistic Regression part of the machine learning assignment.")
```

9.2 Decision Tree Code

```
import pandas as pd

df = pd.read_csv("heart.csv")

df.head()

X = df.drop("target", axis=1)

y = df["target"]

from sklearn.model_selection import train_test_split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

from sklearn.model_selection import GridSearchCV

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)

best_dt = grid.best_estimator_

y_pred_opt = best_dt.predict(X_test)

print("Optimized Accuracy:", accuracy_score(y_test, y_pred_opt))

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(15,10))

plot_tree(best_dt, feature_names=X.columns, class_names=['No Disease', 'Disease'], filled=True)
plt.show()

import pandas as pd

importance = pd.DataFrame({
```

```
    'Feature': X.columns,
    'Importance': best_dt.feature_importances_
}).sort_values(by='Importance', ascending=False)

print(importance)
```

9.3 Random Forest Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, classification_report,confusion_matrix,
ConfusionMatrixDisplay)

df = pd.read_csv('heart.csv')
print(df.shape)       # (1025, 14)
print(df.head())
print(df.info())

# Check class distribution
print(df['target'].value_counts())
df['target'].value_counts().plot(kind='bar', color=['steelblue', 'salmon'])
plt.title('Heart Disease Distribution')
plt.xlabel('Target (0 = No Disease, 1 = Disease)')
plt.ylabel('Count')
plt.show()

# Check for missing values
print(df.isnull().sum())  # All zeros - no missing values!

# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()

# Split features and target
X = df.drop('target', axis=1)
y = df['target']

# Scale features (good practice even for Random Forest)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train/Test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)

rf_model = RandomForestClassifier(
    n_estimators=100,    # number of trees
    max_depth=None,      # let trees grow fully
    random_state=42
)

rf_model.fit(X_train, y_train)
print("Model training complete!")

y_pred = rf_model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

# Detailed report
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
      target_names=['No Disease', 'Heart Disease']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['No Disease', 'Heart Disease'])
disp.plot(cmap='Blues')
plt.title('Random Forest - Confusion Matrix')
```

```
plt.show()

feature_names = df.drop('target', axis=1).columns
importances = rf_model.feature_importances_

# Sort by importance
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 6))
plt.bar(range(len(importances)), importances[indices])
plt.xticks(range(len(importances)),
           [feature_names[i] for i in indices], rotation=45)
plt.title('Random Forest - Feature Importances')
plt.tight_layout()
plt.show()

from sklearn.metrics import precision_score, recall_score, f1_score

# Summary metrics
print('=' * 40)
print('   RANDOM FOREST — RESULTS SUMMARY')
print('=' * 40)
print(f'  Accuracy  : {accuracy_score(y_test, y_pred)*100:.2f}%')
print(f'  Precision : {precision_score(y_test, y_pred)*100:.2f}%')
print(f'  Recall    : {recall_score(y_test, y_pred)*100:.2f}%')
print(f'  F1-Score  : {f1_score(y_test, y_pred)*100:.2f}%')
print('=' * 40)

from sklearn.model_selection import GridSearchCV

# Define the parameter grid to search
param_grid = {
    'n_estimators': [50, 100, 200],        # number of trees
    'max_depth': [None, 5, 10, 20],        # depth of each tree
    'min_samples_split': [2, 5, 10],       # min samples to split a node
    'min_samples_leaf': [1, 2, 4]          # min samples at a leaf node
}

# Grid search with 5-fold cross validation
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

print('Best Parameters:', grid_search.best_params_)
print(f'Best Cross-Validation Accuracy: {grid_search.best_score_*100:.2f}%')

# Evaluate the tuned (optimized) model on the test set
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)

print('=' * 45)
print('  OPTIMIZED RANDOM FOREST — RESULTS')
print('=' * 45)
print(f'  Accuracy  : {accuracy_score(y_test, y_pred_tuned)*100:.2f}%')
print(f'  Precision : {precision_score(y_test, y_pred_tuned)*100:.2f}%')
print(f'  Recall    : {recall_score(y_test, y_pred_tuned)*100:.2f}%')
print(f'  F1-Score  : {f1_score(y_test, y_pred_tuned)*100:.2f}%')
print('=' * 45)

# Compare before and after tuning
print('\nBefore Tuning Accuracy:', f'{accuracy_score(y_test, y_pred)*100:.2f}%')
print('After Tuning Accuracy :', f'{accuracy_score(y_test, y_pred_tuned)*100:.2f}%')

from sklearn.model_selection import cross_val_score, StratifiedKFold

# 10-Fold Stratified Cross Validation on the best model
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

cv_scores = cross_val_score(best_model, X_scaled, y, cv=kfold, scoring='accuracy')
```

```
print('Cross Validation Scores per Fold:')
for i, score in enumerate(cv_scores, 1):
    print(f'  Fold {i:2d}: {score*100:.2f}%')

print(f'\nMean Accuracy : {cv_scores.mean()*100:.2f}%')
print(f'Std Deviation : {cv_scores.std()*100:.2f}%')

plt.figure(figsize=(10, 5))
plt.plot(range(1, 11), cv_scores * 100, marker='o', color='steelblue', linewidth=2, markersize=8)
plt.axhline(y=cv_scores.mean() * 100, color='salmon', linestyle='--', label=f'Mean:
{cv_scores.mean()*100:.2f}%')
plt.title('10-Fold Cross Validation Accuracy')
plt.xlabel('Fold Number')
plt.ylabel('Accuracy (%)')
plt.xticks(range(1, 11))
plt.legend()
plt.tight_layout()
plt.show()

from sklearn.metrics import roc_curve, roc_auc_score

# Get predicted probabilities
y_prob = best_model.predict_proba(X_test)[:, 1]

# Calculate ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

# Plot ROC Curve
plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color='steelblue', linewidth=2,
         label=f'Random Forest (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Classifier')
plt.title('ROC Curve — Random Forest')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

print(f'AUC Score: {auc_score:.4f}')

from sklearn.model_selection import learning_curve

# Generate learning curve data
train_sizes, train_scores, test_scores = learning_curve(
    best_model,
    X_scaled, y,
    cv=5,
    scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

# Calculate mean and std
train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
test_mean  = test_scores.mean(axis=1)
test_std   = test_scores.std(axis=1)

# Plot
plt.figure(figsize=(9, 6))
plt.plot(train_sizes, train_mean * 100, 'o-', color='steelblue', label='Training Accuracy')
plt.fill_between(train_sizes,
                 (train_mean - train_std) * 100,
                 (train_mean + train_std) * 100,
                 alpha=0.15, color='steelblue')

plt.plot(train_sizes, test_mean * 100, 'o-', color='salmon', label='Validation Accuracy')
plt.fill_between(train_sizes,
                 (test_mean - test_std) * 100,
                 (test_mean + test_std) * 100,
                 alpha=0.15, color='salmon')

plt.title('Learning Curve — Random Forest')
plt.xlabel('Training Set Size')
plt.ylabel('Accuracy (%)')
```

```
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

metrics_before = [
    accuracy_score(y_test, y_pred),
    precision_score(y_test, y_pred),
    recall_score(y_test, y_pred),
    f1_score(y_test, y_pred)
]

metrics_after = [
    accuracy_score(y_test, y_pred_tuned),
    precision_score(y_test, y_pred_tuned),
    recall_score(y_test, y_pred_tuned),
    f1_score(y_test, y_pred_tuned)
]

metric_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(metric_names))
width = 0.35

plt.figure(figsize=(10, 6))
bars1 = plt.bar(x - width/2, [m*100 for m in metrics_before], width,
                label='Before Tuning', color='steelblue', edgecolor='black')
bars2 = plt.bar(x + width/2, [m*100 for m in metrics_after], width,
                label='After Tuning', color='salmon', edgecolor='black')

plt.title('Random Forest: Before vs After Hyperparameter Tuning')
plt.ylabel('Score (%)')
plt.xticks(x, metric_names)
plt.ylim(80, 105)
plt.legend()

# Add value labels on bars
for bar in bars1:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
             f'{bar.get_height():.1f}%', ha='center', va='bottom', fontsize=9)
for bar in bars2:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
             f'{bar.get_height():.1f}%', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

print('\nCritical Analysis Summary:')
print('- Hyperparameter tuning using GridSearchCV improved model performance.')
print('- Cross validation confirmed the model generalizes well across different data splits.')
print('- High AUC score indicates strong ability to distinguish between classes.')
print('- Learning curve shows model could benefit from more training data.')
print('- Future work: try feature selection, ensemble with other models, or collect more data.')

import os

results_df = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC Score'],
    'Before Tuning (%)': [
        round(accuracy_score(y_test, y_pred) * 100, 2),
        round(precision_score(y_test, y_pred) * 100, 2),
        round(recall_score(y_test, y_pred) * 100, 2),
        round(f1_score(y_test, y_pred) * 100, 2),
        round(roc_auc_score(y_test, rf_model.predict_proba(X_test)[:, 1]) * 100, 2)
    ],
    'After Tuning (%)': [
        round(accuracy_score(y_test, y_pred_tuned) * 100, 2),
        round(precision_score(y_test, y_pred_tuned) * 100, 2),
        round(recall_score(y_test, y_pred_tuned) * 100, 2),
        round(f1_score(y_test, y_pred_tuned) * 100, 2),
        round(roc_auc_score(y_test, best_model.predict_proba(X_test)[:, 1]) * 100, 2)
    ]
})

results_df.to_csv('random_forest_results.csv', index=False)

print('Results saved to random_forest_results.csv')
print()
print(results_df.to_string(index=False))
```

9.4 Support Vector Machine Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

# Load dataset
data = pd.read_csv("heart.csv")

# Basic preprocessing
data = data.drop_duplicates()
X = data.drop("target", axis=1)
y = data["target"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Base SVM model
svm_model = SVC(kernel="rbf", probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)

# Predictions
svm_pred = svm_model.predict(X_test_scaled)
svm_prob = svm_model.predict_proba(X_test_scaled)[:, 1]

# Evaluation
print("===== SVM Evaluation =====")
print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Precision:", precision_score(y_test, svm_pred))
print("Recall:", recall_score(y_test, svm_pred))
print("F1 Score:", f1_score(y_test, svm_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, svm_prob))
print(classification_report(y_test, svm_pred))

# Confusion matrix
svm_cm = confusion_matrix(y_test, svm_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(svm_cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - SVM")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ROC curve
svm_fpr, svm_tpr, _ = roc_curve(y_test, svm_prob)
plt.figure(figsize=(6, 4))
plt.plot(svm_fpr, svm_tpr, label="SVM")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - SVM")
plt.legend()
plt.show()

# Hyperparameter tuning
param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": ["scale", "auto", 0.1, 0.01],
    "kernel": ["rbf", "linear"]
```

```
}

grid_search_svm = GridSearchCV(
    estimator=SVC(probability=True, random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)

grid_search_svm.fit(X_train_scaled, y_train)

print("Best Parameters:", grid_search_svm.best_params_)
print("Best Cross Validation Score:", grid_search_svm.best_score_)

# Optimized model
best_svm_model = grid_search_svm.best_estimator_
best_svm_pred = best_svm_model.predict(X_test_scaled)
best_svm_prob = best_svm_model.predict_proba(X_test_scaled)[:, 1]

print("===== Optimized SVM Evaluation =====")
print("Accuracy:", accuracy_score(y_test, best_svm_pred))
print("Precision:", precision_score(y_test, best_svm_pred))
print("Recall:", recall_score(y_test, best_svm_pred))
print("F1 Score:", f1_score(y_test, best_svm_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, best_svm_prob))

# Save results
svm_results = pd.DataFrame([{
    "Model": "Support Vector Machine",
    "Accuracy": accuracy_score(y_test, best_svm_pred),
    "Precision": precision_score(y_test, best_svm_pred),
    "Recall": recall_score(y_test, best_svm_pred),
    "F1 Score": f1_score(y_test, best_svm_pred),
    "ROC-AUC": roc_auc_score(y_test, best_svm_prob)
}])

svm_results.to_csv("svm_results.csv", index=False)
print("SVM results saved successfully.")
```

9.5 Main Notebook Code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load model result files
lr_results = pd.read_csv("logistic_regression_metrics.csv")
dt_results = pd.read_csv("decision_tree_results.csv")
rf_results = pd.read_csv("random_forest_results.csv")

# Add SVM results file when available
# svm_results = pd.read_csv("svm_results.csv")

comparison_df = pd.DataFrame([
    {
        "Model": "Logistic Regression",
        "Accuracy": lr_results.loc[0, "Accuracy"],
        "Precision": lr_results.loc[0, "Precision"],
        "Recall": lr_results.loc[0, "Recall"],
        "F1 Score": lr_results.loc[0, "F1 Score"]
    },
    {
        "Model": "Decision Tree",
        "Accuracy": dt_results.loc[0, "Test Accuracy"],
        "Precision": dt_results.loc[0, "Test Precision"],
        "Recall": dt_results.loc[0, "Test Recall"],
        "F1 Score": dt_results.loc[0, "Test F1"]
    },
    {
        "Model": "Random Forest",
        "Accuracy": 1.00,
        "Precision": 1.00,
        "Recall": 1.00,
        "F1 Score": 1.00
    }
    # Uncomment after adding SVM results
    # ,
    # {
    #     "Model": "Support Vector Machine",
    #     "Accuracy": svm_results.loc[0, "Accuracy"],
    #     "Precision": svm_results.loc[0, "Precision"],
    #     "Recall": svm_results.loc[0, "Recall"],
    #     "F1 Score": svm_results.loc[0, "F1 Score"]
    # }
])

print(comparison_df)

comparison_df.to_csv("model_comparison.csv", index=False)

plt.figure(figsize=(8, 5))
plt.bar(comparison_df["Model"], comparison_df["Accuracy"])
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.xlabel("Machine Learning Models")
plt.xticks(rotation=15)
plt.ylim(0, 1.05)
plt.show()

best_model = comparison_df.loc[comparison_df["Accuracy"].idxmax()]
print("Best Model:")
print(best_model)
```